C4-2a : Create Database and Tables

Basic SQL - Using DB Browser for SQLite

SQL and SQLite



SQL stands for **S**tructured **Q**uery **L**anguage

The various versions are MySQL, Microsoft Sybase, Oracle, SAP, PostgreSQL

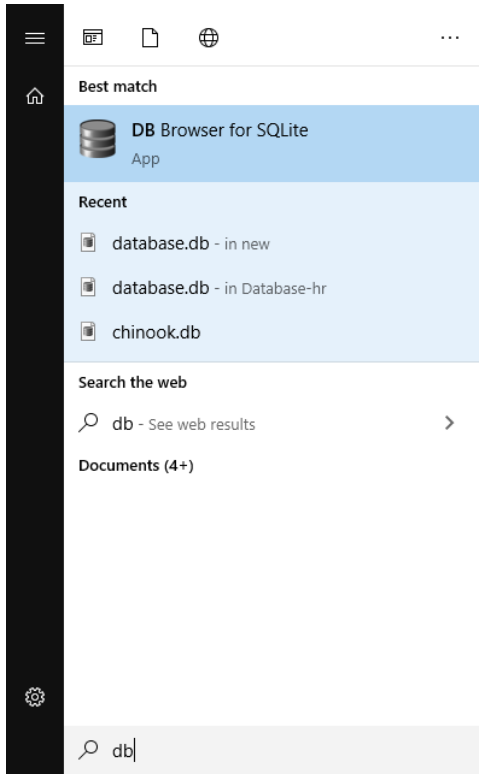
SQL Server is a client-server database management system (DBMS) that consists of a database server and client applications that connect to the server.

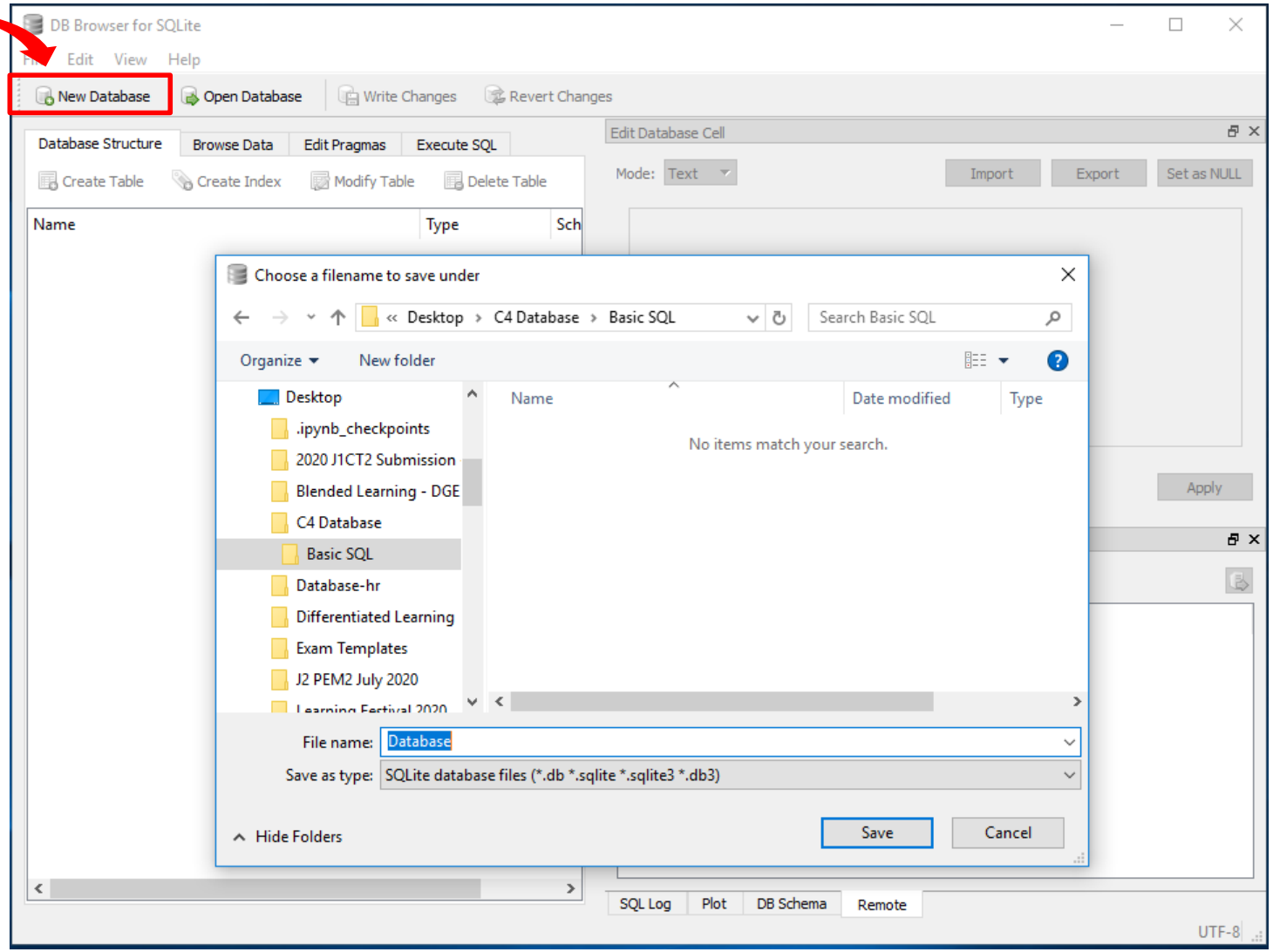
SQLite is a serverless DBMS that can be embedded directly into an application.

All DBMS support the major commands for **C**reate, **R**ead, **U**pdate and **D**elete functions (CRUD)

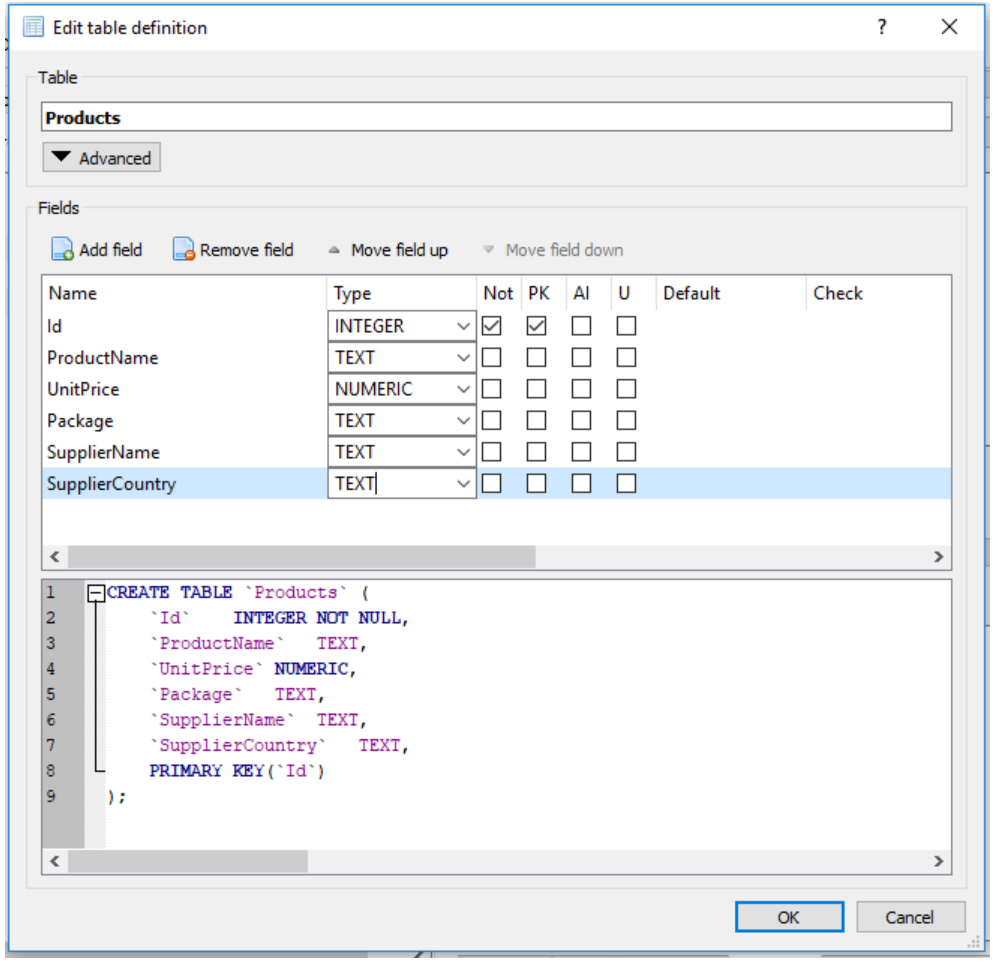
DB Browser for SQLite

- Launch DB Browser
- Create a new database and save it as `Database.db`





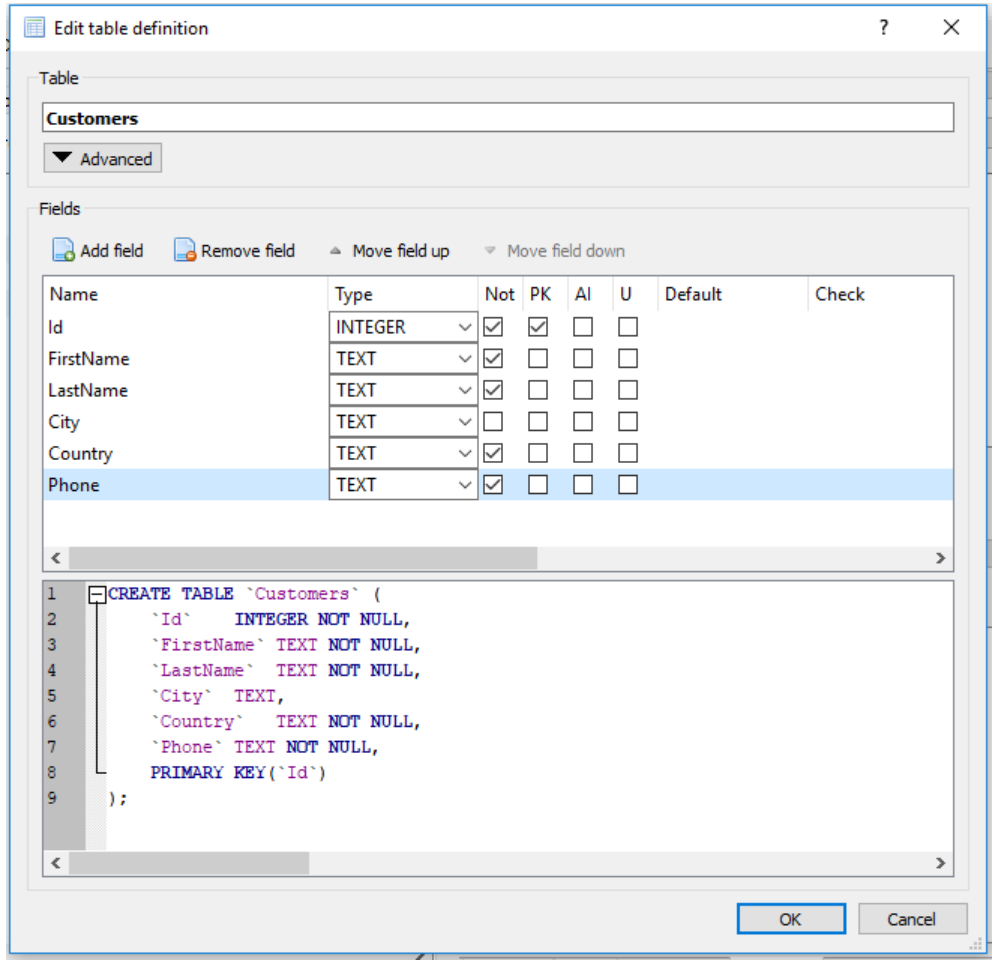
Create Table Products



Products
Id
ProductName
UnitPrice
Package
SupplierName
SupplierCountry

```
CREATE TABLE Products (
        Id                      INTEGER NOT NULL,
        ProductName             TEXT,
        UnitPrice               NUMERIC,
        Package                 TEXT,
        SupplierName            TEXT,
        SupplierCountry         TEXT,
        PRIMARY KEY(Id)
);
```

Create Table Customers

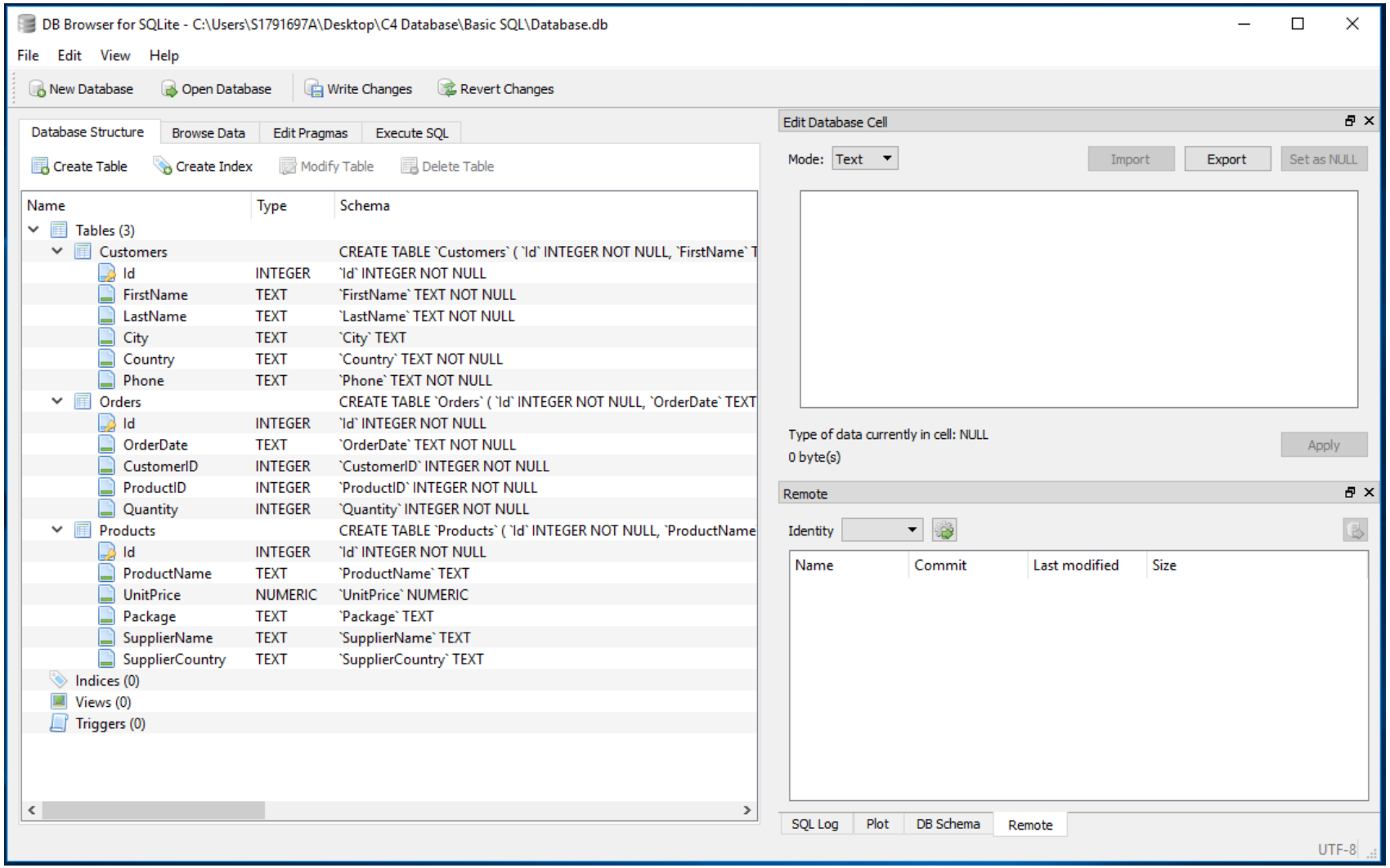


Customers
Id
FirstName
LastName
City
Country
Phone

```
CREATE TABLE Customers (
        Id              INTEGER NOT NULL,
        FirstName       TEXT NOT NULL,
        LastName        TEXT NOT NULL,
        City            TEXT,
        Country         TEXT NOT NULL,
        Phone           TEXT NOT NULL,
        PRIMARY KEY(Id)
);
```

Database Structure – Table Schema

- Copy the create statements, paste them in a notepad and save it as "`Create Tables.sql`"

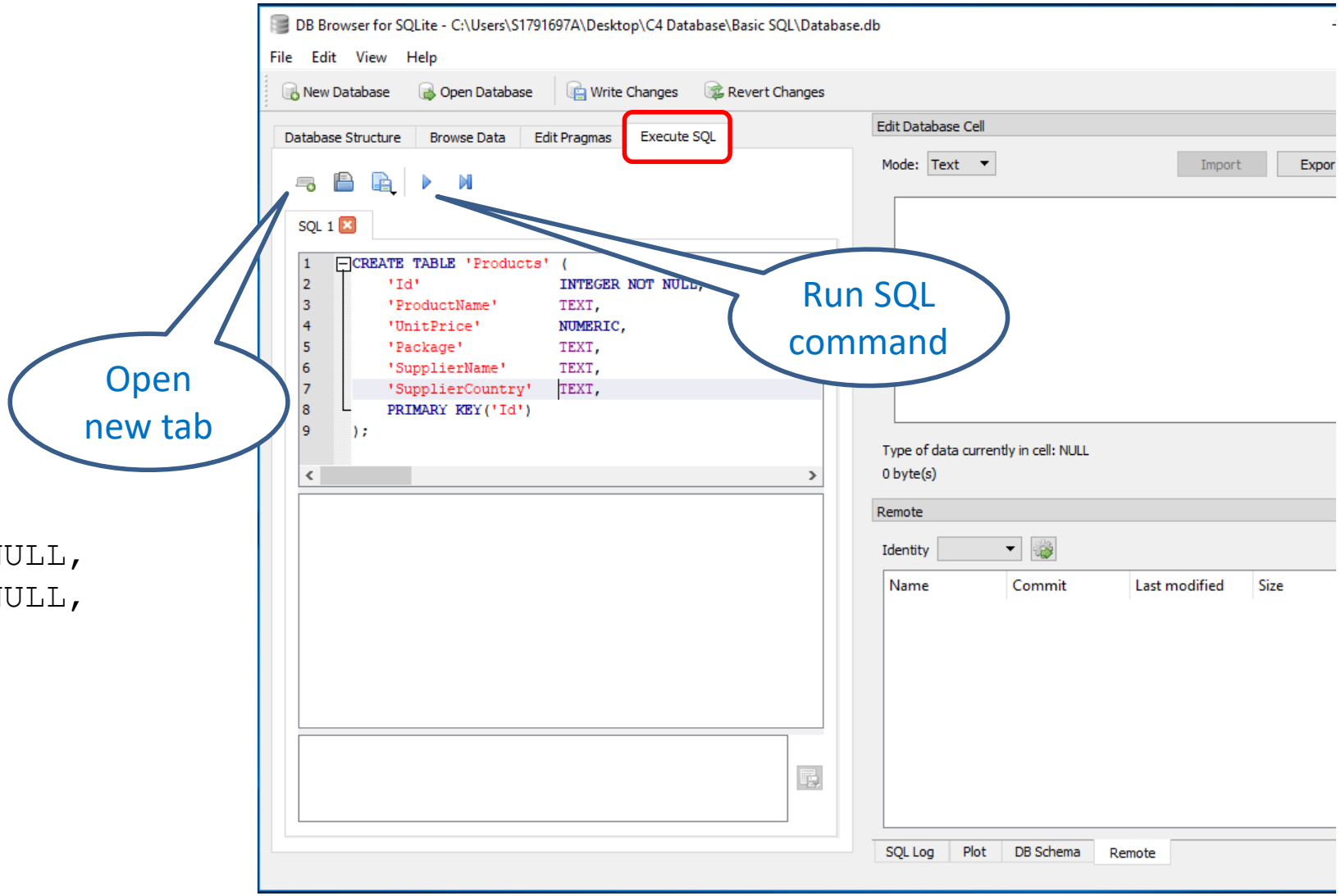


Execute SQL

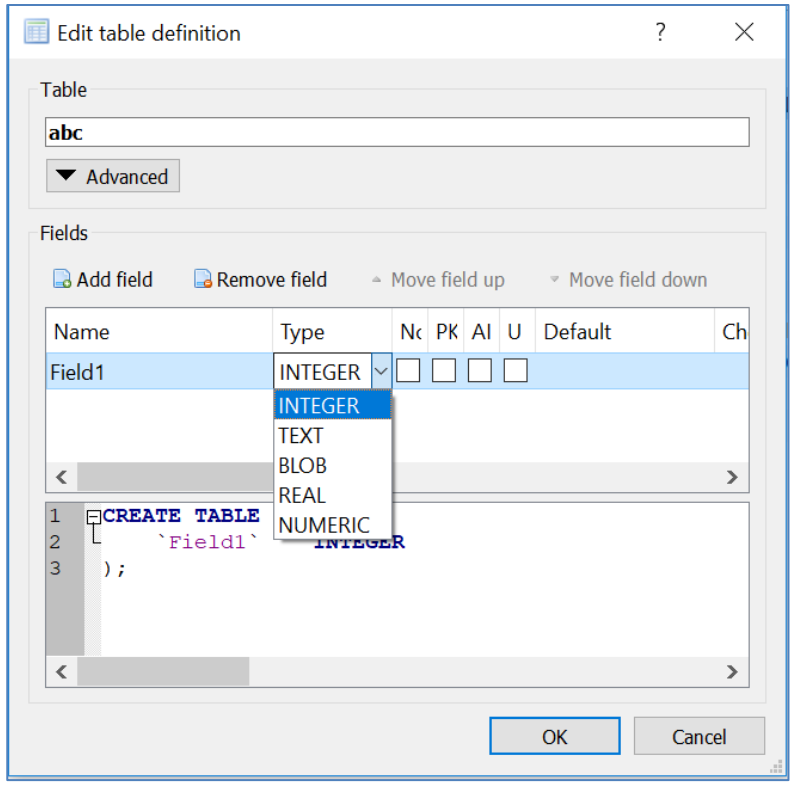
Create Tables using SQL Commands

```
CREATE TABLE Products (
Id              INTEGER NOT NULL,
ProductName     TEXT,
UnitPrice               REAL,
Package         TEXT,
SupplierName    TEXT,
SupplierCountry         TEXT,
PRIMARY KEY(Id)
);

CREATE TABLE Customers (
Id              INTEGER NOT NULL,
FirstName               TEXT NOT NULL,
LastName                TEXT NOT NULL,
City            TEXT,
Country         TEXT NOT NULL,
Phone           TEXT NOT NULL,
PRIMARY KEY(Id)
);
```



Data Types



Data Type	Description
INTEGER	Whole number
TEXT	Text string : CHAR(n), VARCHAR(n)
BLOB	Binary Data Type Eg. Image file
REAL	Decimal (or Floating point) value
NUMERIC	Any other type not listed above. Eg. Boolean, DateTime …

VARCHAR is variable length, while **CHAR** is fixed length

Boolean values can also be stored as INTEGER, e.g. 0 (False) and 1 (True).
DateTime can also be stored as TEXT, e.g. in the 'YYYY-MM-DD' and 'HH:MM:SS.SSS' formats.

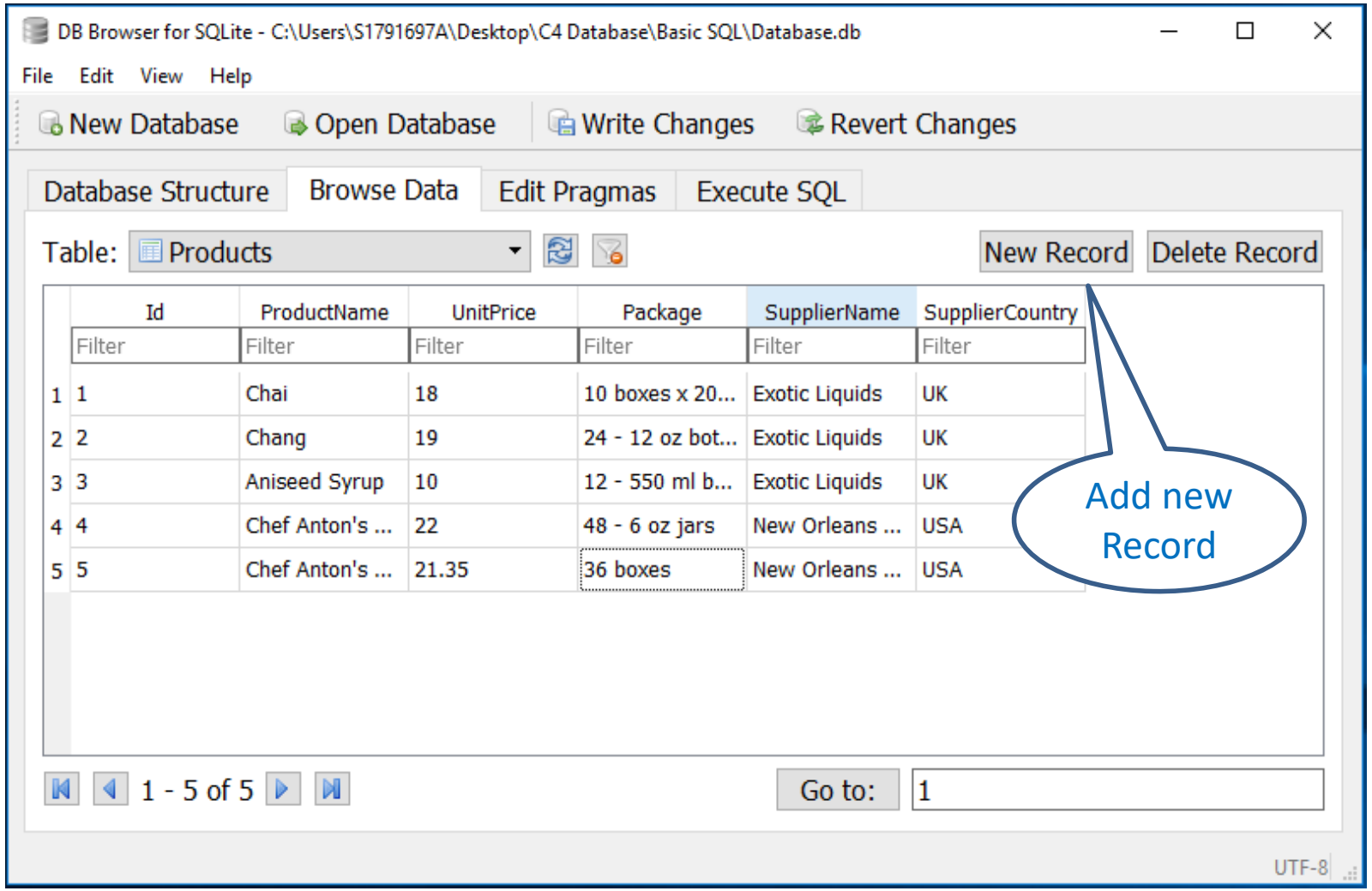
Note: There is no need to go into details about the *Type Affinity* in SQLite (https://www.sqlite.org/datatype3.html)

C4-2b: *Insert Data into Tables*

Basic SQL - Using DB Browser for SQLite

Input data using DB Browser

- `Id` field will be populated by auto-increment generated values
- A field without a value is saved with a NULL



Execute SQL

Insert one row of data into table

```
INSERT INTO Products
      (ProductName, UnitPrice, Package, SupplierName, SupplierCountry)
VALUES
      ('Chai', 18, '10 boxes x 20 bags', 'Exotic Liquids', 'UK');
```

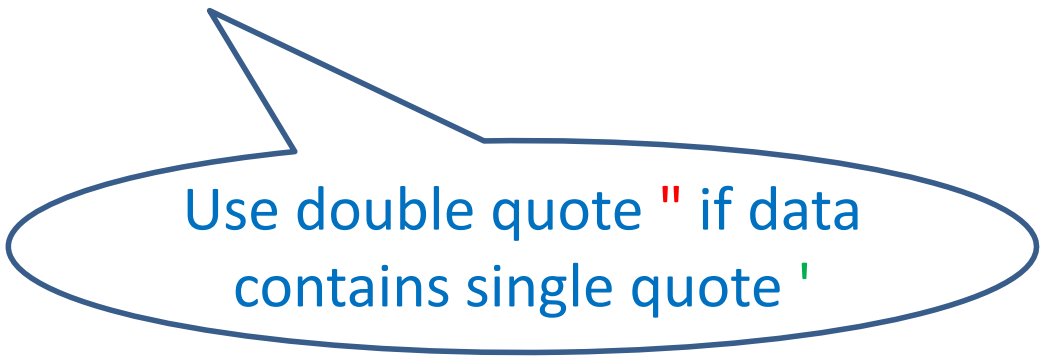
Id	ProductName	UnitPrice	Package	SupplierName	SupplierCountry
1	Chai	18	10 boxes x 20 bags	Exotic Liquids	UK

- `Id` field will be populated by auto-increment generated values

Execute SQL

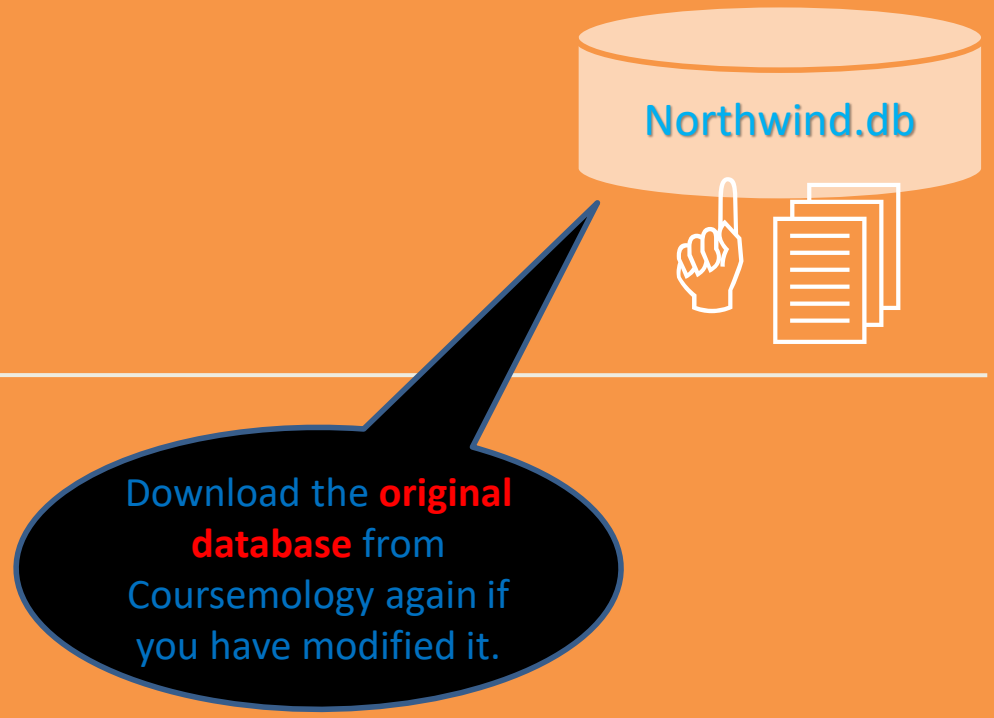
Insert multiple rows of data into table

```
INSERT INTO Products
      (ProductName, UnitPrice, Package, SupplierName, SupplierCountry)
VALUES
      ('Chang', 19, '24 - 12 oz bottles', 'Exotic Liquids', 'UK'),
      ('Aniseed Syrup', 10, '12 - 550ml bottles', 'Exotic Liquids', 'UK'),
      ("Chef Anton's Cajun Seasoning", 22, '48 - 6 oz jars', 'New Orleans
Cajun Delights', 'USA'),
      ("Chef Anton's Gumbo Mix", 21.35, '36 boxes', 'New Orleans Cajun
Delights', 'USA');
```



C4-2c: Basic Query

Basic SQL - Using DB Browser for SQLite



Execute SQL

Using SELECT

- View the data in all the fields in the table

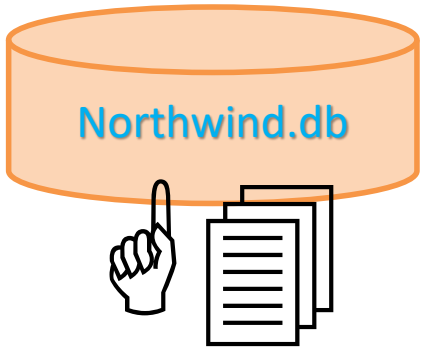
```
SELECT * FROM Products;
```

- View the data in some of the fields in the table

```
SELECT Id, ProductName, UnitPrice FROM Products;
```

- View distinct data in a field

```
SELECT DISTINCT SupplierName FROM Products;
```



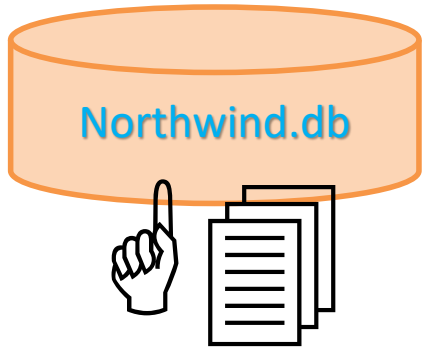
Execute SQL

Using WHERE

- Select all the customers from 'Mexico':

```
SELECT * FROM Customers WHERE Country = 'Mexico';
```

Operator	Description
=	Equal
>	Greater than
<	Lesser than
>=	Greater than or equal
<=	Lesser than or equal
<>	Not equal



Execute SQL

Using WHERE with other Operators

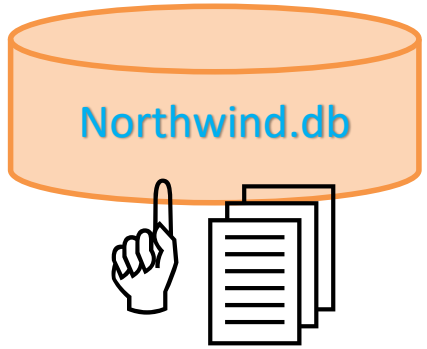
Operator	Description	Example
BETWEEN	Between a certain range	`SELECT ProductName, UnitPrice FROM Products WHERE UnitPrice BETWEEN 10 AND 20;`
LIKE	Search for a pattern	`SELECT FirstName FROM Customers WHERE FirstName LIKE '%ja%'`
IN	Multiple OR conditions	`SELECT * FROM Customers WHERE Country IN ('Germany', 'France', 'UK')`

Wildcard Character

Wildcard Characters % and _:

bl% finds bl, black, blue and blur

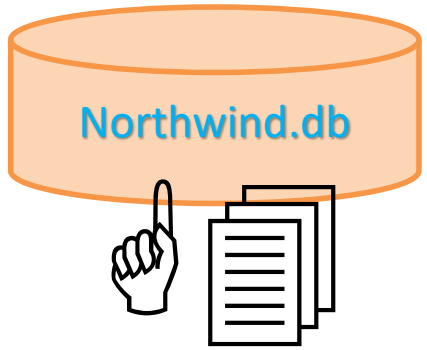
h_t finds hot, hat and hit



Execute SQL

Using WHERE with AND, OR, and NOT Operators

Operator	Example
AND	`SELECT * FROM Customers` `WHERE Country='Germany' AND City='Berlin'`
OR	`SELECT * FROM Customers` `WHERE Country='Germany' OR Country='Spain'`
NOT	`SELECT * FROM Customers` `WHERE NOT Country='Germany'`



Combining AND, OR, and NOT:

```
SELECT * FROM Customers
WHERE Country='USA' AND (City='Seattle' OR City='Portland');
```

Execute SQL

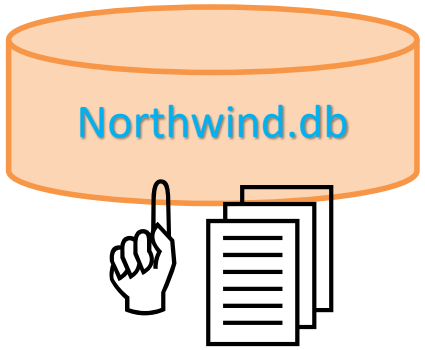
Using ORDER BY and LIMIT

```
SELECT ProductName, UnitPrice FROM Products
ORDER BY UnitPrice DESC LIMIT 10;

SELECT FirstName, Country FROM Customers
ORDER BY Country ASC, FirstName DESC;
```

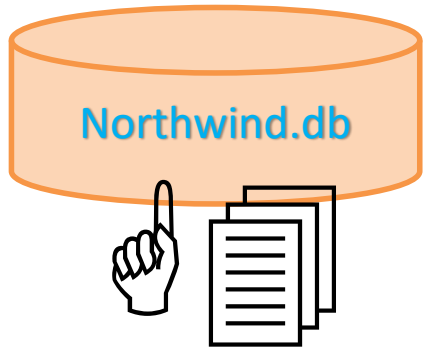
Top 10 most expensive products

The ORDER BY sorts the records in ascending order by default.



Execute SQL

Using GROUP BY



Execute the following SQL statements, one by one, and observe the outputs:

```
SELECT ProductName, SupplierCountry FROM Products;

SELECT ProductName, SupplierCountry FROM Products
GROUP BY SupplierCountry;

SELECT COUNT(ProductName), SupplierCountry FROM Products
GROUP BY SupplierCountry;

SELECT COUNT(ProductName), SupplierCountry FROM Products
GROUP BY SupplierCountry
ORDER BY COUNT(ProductName) DESC;
```

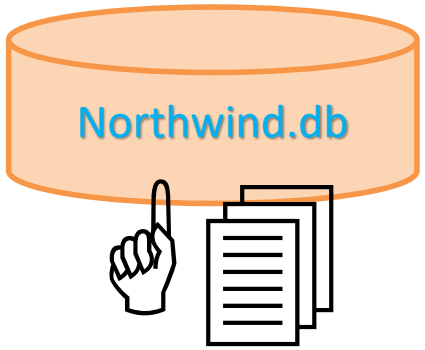
COUNT is an Aggregate Function

Execute SQL

Aggregate Functions

- COUNT, AVG, SUM, MAX, MIN

```
SELECT COUNT(Id) FROM Products;
SELECT AVG(UnitPrice) FROM Products;
SELECT SUM(Quantity) FROM Orders;
SELECT MAX(UnitPrice) FROM Products;
SELECT MIN(UnitPrice) FROM Products;
```



Execute SQL

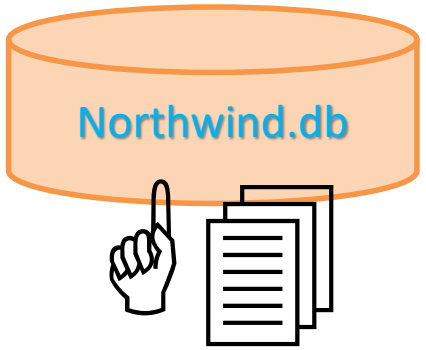
Using HAVING

```
SELECT COUNT(Id), Country FROM Customers
GROUP BY Country
HAVING COUNT(Id) > 5
ORDER BY COUNT(Id) DESC;
```

Note: SQL WHERE clause cannot be used with aggregate functions.

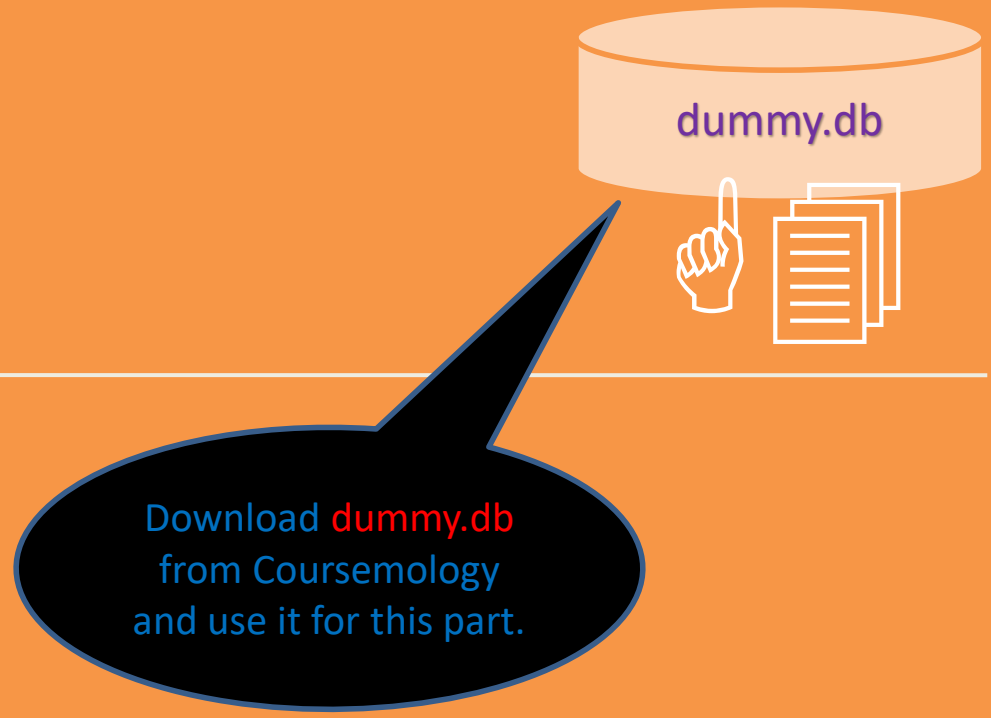
OUTPUT:

COUNT(Id)	Country
13	USA
11	France
11	Germany
9	Brazil
7	UK



C4-2d: Update Data

Basic SQL - Using DB Browser for SQLite



Execute SQL

Using UPDATE and SET

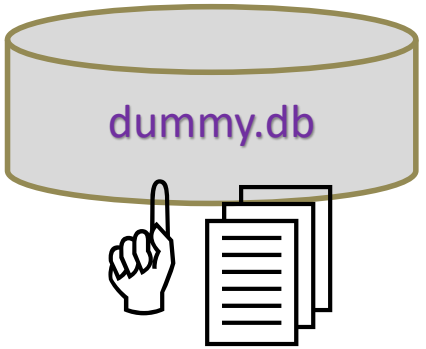
```
UPDATE Customers SET Country='Singapore'
WHERE FirstName='Thomas' and LastName='Hardy';

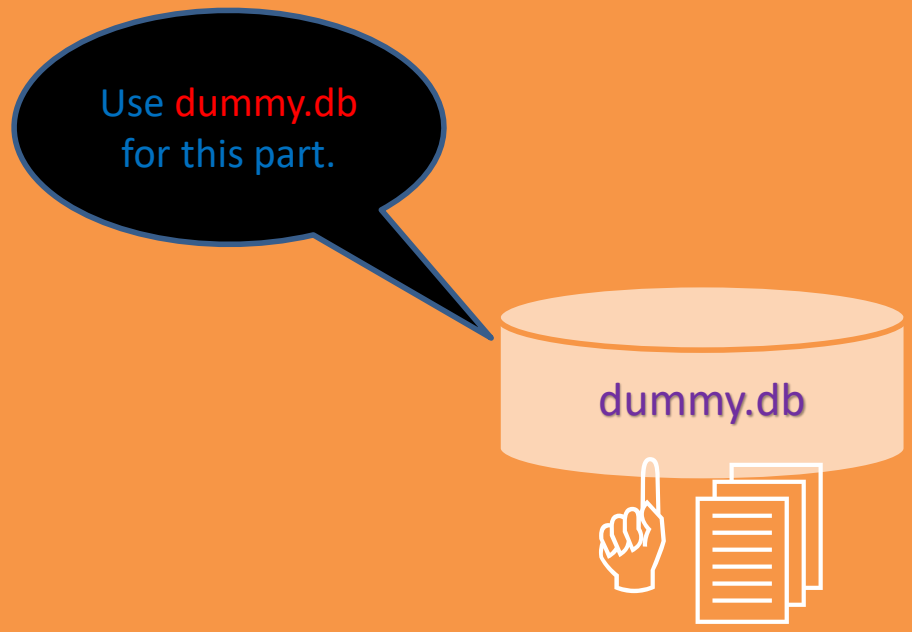
UPDATE Customers SET Country='Singapore';

UPDATE Orders SET Quantity = Quantity + 5
WHERE Id=7;
```

THIS WILL CHANGE EVERYONE's COUNTRY TO 'Singapore' !!!

This will increase the Quantity by 5.



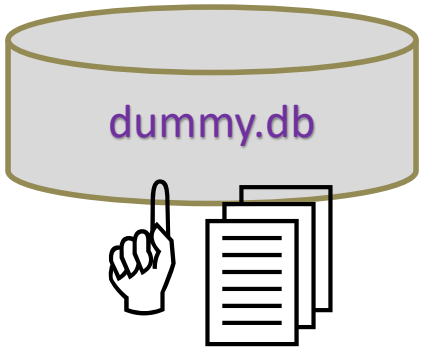


C4-2e: Delete Data and Drop Table

Basic SQL - Using DB Browser for SQLite

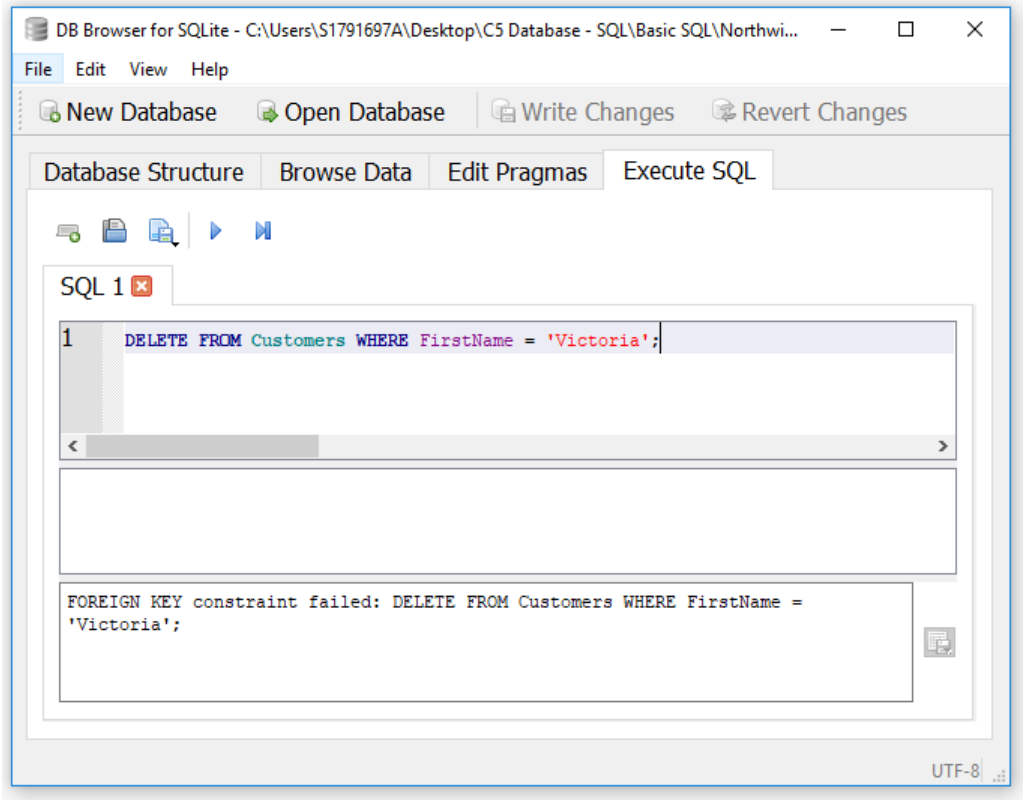
Execute SQL

Delete Record(s) in a Table




```
DELETE FROM Customers WHERE FirstName = 'Victoria';
```

Note: The Foreign Key constraint will prevent the deletion because Victoria Ashworth's Id is referred to in the Orders table.




```
DELETE FROM Customers;
```

DANGER: This will delete all the records in the table but the Table will still remain.

Execute SQL

Drop a Table

```
DROP TABLE Customers;
```

DANGER: This will delete the Table.

Note: The Foreign Key constraint will prevent this if Foreign Key is defined when creating the relating table(s).

```
DROP DATABASE Northwind;
```

NOTE: There is no drop database in SQLite.



